CredVault - API Reference Documentation

> **Version:** v1.0
> **Last Updated:** April 2026
> **Base URL:** `http://localhost:5006` (API Gateway)
> **API Versioning:** URL-based (`/api/v1/`)

Table of Contents

1. Architecture Overview
2. API Standards & Conventions
3. Authentication & Authorization
4. Identity Service API
5. Card Service API
6. Billing Service API
7. Payment Service API
8. Notification Service API
9. Error Handling
10. Rate Limiting & Throttling
11. Health Checks
12. Frontend Integration Guide
13. Core Business Flows
14. Inter-Service Event Reference
15. Appendix

1. Architecture Overview

1.1 System Topology

CredVault follows a **microservices architecture** with an **API Gateway pattern**. All client requests route through the Ocelot API Gateway, which proxies requests to the appropriate downstream service based on URL path prefixes. Services communicate asynchronously via RabbitMQ for event-driven workflows and saga orchestration.

```
┌─────────────────┐
│  Angular Client  │  (sessionStorage JWT, auth interceptor)
└─────────────────┘
         │ HTTPS
         ▼
┌─────────────────┐
│  Ocelot Gateway  │  :5006 — Single entry point, route-by-prefix
└─────────────────┘
         │ HTTP (internal network)
    ┌────┴────┬────┬────┬────┬────┐
    ▼    ▼    ▼    ▼    ▼    ▼
 ┌────┐ ┌────┐ ┌────┐ ┌────┐ ┌────┐
 │ 5001│ │ 5002│ │ 5003│ │ 5004│ │ 5005│
 │Ident│ │Card│ │Bill│ │Pay │ │Notif│
 └────┘ └────┘ └────┘ └────┘ └────┘
    ▼      ▼      ▼      ▼      ▼
  [DB]   [DB]   [DB]   [DB]   [DB]

         ┌─────────────────┐
         │    RabbitMQ     │  Async: events, saga orchestration
         │   :5672/:15672  │
         └─────────────────┘
```

1.2 Service Routing Table

All routes are prefixed with `/api/v1/`. The Gateway strips the service-specific prefix and forwards to the internal service.

Gateway Route Prefix	Destination Service	Internal Port	Database
`/api/v1/identity/`	Identity Service	5001	`credvault_identity`
`/api/v1/cards/`	Card Service	5002	`credvault_cards`
`/api/v1/issuers/`	Card Service	5002	`credvault_cards`
`/api/v1/billing/`	Billing Service	5003	`credvault_billing`
`/api/v1/payments/`	Payment Service	5004	`credvault_payments`
`/api/v1/wallets/`	Payment Service	5004	`credvault_payments`
`/api/v1/notifications/`	Notification Service	5005	`credvault_notifications`

1.3 Service Responsibility Matrix

Capability	Identity	Card	Billing	Payment	Notification
User Registration	Primary	—	—	—	Email
Authentication (JWT)	Primary	Validates	Validates	Validates	—
Google OAuth SSO	Primary	—	—	—	—
Card CRUD	—	Primary	—	Deduction (saga)	Email

Capability	Identity	Card	Billing	Payment	Notification
Bill Generation	—	—	Primary	—	—
Statement Creation	—	—	Primary	—	—
Rewards (Earn/Redeem)	—	—	Primary	Trigger (saga)	—
Payment Initiation	—	—	—	Primary	OTP Email
Wallet Management	—	—	—	Primary	—
Audit & Notification Logs	—	—	—	—	Primary

2. API Standards & Conventions

2.1 Base URLs

Environment	URL	Notes
Development	`http://localhost:5006`	All services via Gateway
Docker	`http://gateway:5006`	Internal Docker network
Production	`https://api.credvault.com`	TLS terminated at reverse proxy

> **Rule:** Clients MUST always communicate through the API Gateway. Direct service access is restricted to the internal Docker network.

2.2 Required Headers

Header	Required	Value	Description
`Authorization`	Yes (protected)	`Bearer <JWT>`	JWT issued by Identity Service
`Content-Type`	Yes (POST/PUT/PATCH)	`application/json`	Request body format
`Accept`	No	`application/json`	Expected response format
`X-Trace-Id`	No	`<GUID>`	Client-provided correlation ID for distributed tracing

2.3 Standard Response Envelope

Every API response is wrapped in a consistent `ApiResponse<T>` envelope:

Success Response

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "email": "user@example.com"
  },
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

Error Response

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    "Email is required",
    "Password must be at least 8 characters"
  ],
  "traceId": "0HN12345678ABC"
}
```

Paginated Response

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {
    "items": [
      { "id": "...", "email": "user1@example.com" },
      { "id": "...", "email": "user2@example.com" }
    ],
    "totalCount": 150,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 8
  },
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

2.4 HTTP Status Codes

Code	Category	Usage
`200`	Success	Successful GET, PUT, PATCH, or DELETE

Code	Category	Usage
`201`	Success	Resource created via POST
`400`	Client Error	Validation failure, malformed request, or business rule violation
`401`	Security	Missing, expired, or invalid JWT token
`403`	Security	Insufficient role/permissions
`404`	Client Error	Requested resource not found
`409`	Client Error	Conflict (e.g., duplicate email, duplicate card)
`422`	Client Error	Business logic failure (e.g., insufficient wallet balance)
`500`	Server Error	Unhandled internal error
`503`	Server Error	Downstream service or database unavailable

2.5 Pagination

List endpoints support pagination via query parameters:

Parameter	Type	Default	Max	Description
`pageNumber`	int	1	—	Page number (1-indexed)
`pageSize`	int	20	100	Items per page

Example:

```
GET /api/v1/identity/users?pageNumber=2&pageSize=10
```

2.6 Date & Time Format

- All timestamps are in **ISO 8601 UTC format**: `2026-04-30T14:30:00Z`
- Currency amounts use **2 decimal places**: `1500.00`
- Currency code: `INR` (Indian Rupee)

2.7 ID Format

- All identifiers are **GUIDs/UUIDs** (v4): `3fa85f64-5717-4562-b3fc-2c963f66afa6`

3. Authentication & Authorization

3.1 Authentication Methods

Method	Endpoint	Description
Email/Password	`POST /api/v1/identity/auth/login`	Standard credential-based login
Google OAuth	`POST /api/v1/identity/auth/google`	Passwordless login via Google IdToken

3.2 JWT Token Specification

Configuration:

Claim	Value
Issuer (`iss`)	`CredVault`
Audience (`aud`)	`CredVaultClient`
Expiry	60 minutes from issue
Signing Algorithm	HS256
Clock Skew Tolerance	30 seconds

Token Payload:

```json
{
  "sub": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "email": "user@example.com",
  "name": "John Doe",
  "role": "user",
  "iat": 1714486200,
  "exp": 1714489800,
  "iss": "CredVault",
  "aud": "CredVaultClient"
}
```

3.3 Token Lifecycle

```
Registration → Email OTP Verification → JWT Issued → 60 min validity
                                                  |
                                        Token expires → 401 Unauthorized
                                                  |
                                        Client auto-logs out → Redirect to /login
```

3.4 RBAC Permission Matrix

Endpoint Pattern	Public	User	Admin
`/auth/register`	Yes	—	—
`/auth/login`	Yes	—	—
`/auth/google`	Yes	—	—
`/auth/verify-email-otp`	Yes	—	—
`/auth/resend-verification`	Yes	—	—
`/auth/forgot-password`	Yes	—	—
`/auth/reset-password`	Yes	—	—
`/users/me`	—	Yes	Yes
`/users` (list)	—	—	Yes
`/users/{id}/status`	—	—	Yes
`/users/{id}/role`	—	—	Yes
`/users/stats`	—	—	Yes
`/cards`	—	Yes	Yes
`/cards/admin`	—	—	Yes
`/issuers` (read)	—	Yes	Yes
`/issuers` (write)	—	—	Yes
`/billing/bills`	—	Yes	Yes
`/billing/bills/admin/*`	—	—	Yes
`/billing/statements`	—	Yes	Yes
`/billing/rewards/*` (read)	—	Yes	Yes
`/billing/rewards/tiers` (write)	—	—	Yes
`/payments/*`	—	Yes	Yes
`/wallets/*`	—	Yes	Yes
`/notifications/logs`	—	—	Yes
`/notifications/audit`	—	—	Yes

3.5 OTP Configuration

OTP Type	Length	Expiry	Use Case
Email Verification	6 digits	10 minutes	Post-registration verification

OTP Type	Length	Expiry	Use Case
Password Reset	6 digits	10 minutes	Forgot password flow
Payment Verification	6 digits	5 minutes	Bill payment 2FA

4. Identity Service API

Base Path: `/api/v1/identity`
Port: 5001
Database: `credvault_identity`

4.1 Authentication Endpoints

4.1.1 Register User

Register a new user account. The user is created with `PendingVerification` status and an email with OTP is sent.

```
POST /api/v1/identity/auth/register
```

Request Body:

```
{
  "fullName": "John Doe",
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Validation Rules:

Field	Type	Required	Constraints
`fullName`	string	Yes	Min 2 characters, max 200
`email`	string	Yes	Valid email format, unique across system
`password`	string	Yes	Min 8 chars, 1 uppercase, 1 lowercase, 1 digit

Response (201 Created):

```
{
  "success": true,
  "message": "Registration successful. Please verify your email with the OTP sent to john@example.com",
  "data": {
    "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "email": "john@example.com",
    "status": "PendingVerification"
  },
  "errors": []
}
```

Error Responses:

Status	Condition
400	Validation errors (invalid email, weak password)
409	Email already registered

4.1.2 Login (Email/Password)

Authenticate with email and password. Returns a JWT token on success.

```
POST /api/v1/identity/auth/login
```

Request Body:

```
{
  "email": "john@example.com",
  "password": "SecurePass123!"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "email": "john@example.com",
      "fullName": "John Doe",
      "role": "user",
      "status": "active"
    }
  },
  "errors": []
}
```

Error Responses:

Status	Condition
400	Validation errors
401	Invalid email or password
403	Account not verified (PendingVerification)
403	Account suspended

> **Security Note:** Error messages for invalid credentials are intentionally generic to prevent user enumeration.

4.1.3 Google OAuth Login

Authenticate using a Google IdToken. Creates a new account if the user doesn't exist.

```
POST /api/v1/identity/auth/google
```

Request Body:

```
{
  "idToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6..."
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "email": "john@gmail.com",
      "fullName": "John Doe",
      "role": "user",
      "status": "active"
    },
    "isNewUser": false
  },
  "errors": []
}
```

Flow:

1. Frontend obtains Google IdToken via Google Sign-In SDK
2. IdToken sent to this endpoint
3. Identity Service validates IdToken against Google's public keys
4. If user exists → login; if not → create account with `Active` status (no email verification needed)
5. JWT returned

Error Responses:

Status	Condition
400	Invalid or missing IdToken
401	Google IdToken validation failed

4.1.4 Verify Email OTP

Verify the email address using the 6-digit OTP sent during registration.

```
POST /api/v1/identity/auth/verify-email-otp
```

Request Body:

```
{
  "email": "john@example.com",
  "otp": "123456"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Email verified successfully",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "email": "john@example.com",
      "fullName": "John Doe",
      "role": "user",
      "status": "active"
    }
  },
  "errors": []
}
```

Error Responses:

Status	Condition
400	Invalid OTP format
401	Invalid or expired OTP
404	User not found

4.1.5 Resend Verification OTP

Request a new OTP if the previous one expired or was lost.

```
POST /api/v1/identity/auth/resend-verification
```

Request Body:

```
{
  "email": "john@example.com"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Verification OTP has been resent to john@example.com",
  "data": null,
  "errors": []
}
```

Error Responses:

Status	Condition
400	Email already verified
404	User not found

4.1.6 Forgot Password

Request a password reset OTP.

```
POST /api/v1/identity/auth/forgot-password
```

Request Body:

```
{
  "email": "john@example.com"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Password reset OTP has been sent to john@example.com",
  "data": null,
  "errors": []
}
```

4.1.7 Reset Password

Reset the password using the OTP received from the forgot password flow.

```
POST /api/v1/identity/auth/reset-password
```

Request Body:

```
{
  "email": "john@example.com",
  "otp": "654321",
  "newPassword": "NewSecurePass456!"
}
```

Validation Rules:

Field	Type	Required	Constraints
`email`	string	Yes	Valid email format
`otp`	string	Yes	6-digit code
`newPassword`	string	Yes	Min 8 chars, 1 uppercase, 1 lowercase, 1 digit

Response (200 OK):

```
{
  "success": true,
  "message": "Password reset successful",
  "data": null,
  "errors": []
}
```

Error Responses:

Status	Condition
400	Validation errors
401	Invalid or expired OTP
404	User not found

4.2 User Profile Endpoints

4.2.1 Get Current Profile

Retrieve the authenticated user's profile information.

```
GET /api/v1/identity/users/me
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "email": "john@example.com",
    "fullName": "John Doe",
    "role": "user",
    "status": "active",
    "isEmailVerified": true,
    "createdAt": "2026-01-15T10:30:00Z",
    "updatedAt": "2026-04-20T14:00:00Z"
  },
  "errors": []
}
```

4.2.2 Update Profile

Update the authenticated user's profile information.

```
PUT /api/v1/identity/users/me
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "fullName": "John A. Doe"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Profile updated successfully",
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "email": "john@example.com",
    "fullName": "John A. Doe",
    "role": "user",
    "status": "active",
    "updatedAt": "2026-04-30T12:00:00Z"
  },
  "errors": []
}
```

4.2.3 Change Password

Change the authenticated user's password.

```
PUT /api/v1/identity/users/me/password
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "oldPassword": "SecurePass123!",
  "newPassword": "NewSecurePass789!"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Password changed successfully",
  "data": null,
  "errors": []
}
```

Error Responses:

Status	Condition
400	Validation errors
401	Incorrect old password

4.3 Admin User Management Endpoints

4.3.1 List All Users

Retrieve a paginated list of all users in the system.

```
GET /api/v1/identity/users?pageNumber=1&pageSize=20
Authorization: Bearer <JWT> (Admin)
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number (1-indexed)
`pageSize`	int	20	Items per page (max 100)

Response (200 OK):

```
{
  "success": true,
  "message": "Users retrieved successfully",
  "data": {
    "items": [
      {
        "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "email": "john@example.com",
        "fullName": "John Doe",
        "role": "user",
        "status": "active",
        "isEmailVerified": true,
        "createdAt": "2026-01-15T10:30:00Z"
      }
    ],
    "totalCount": 150,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 8
  },
  "errors": []
}
```

4.3.2 Get User by ID

Retrieve a specific user's details.

```
GET /api/v1/identity/users/{userId}
Authorization: Bearer <JWT> (Admin)
```

Path Parameters:

Parameter	Type	Description
`userId`	GUID	The user's unique identifier

Response (200 OK):

```
{
  "success": true,
  "message": "User retrieved successfully",
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "email": "john@example.com",
    "fullName": "John Doe",
    "role": "user",
    "status": "active",
    "isEmailVerified": true,
    "createdAt": "2026-01-15T10:30:00Z",
    "updatedAt": "2026-04-20T14:00:00Z"
  },
  "errors": []
}
```

4.3.3 Update User Status

Change a user's account status (Active / Suspended).

```
PUT /api/v1/identity/users/{userId}/status
Authorization: Bearer <JWT> (Admin)
```

Request Body:

```
{
  "status": "suspended"
}
```

Allowed Values: `active` , `suspended`

Response (200 OK):

```
{
  "success": true,
  "message": "User status updated to suspended",
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "status": "suspended"
  },
  "errors": []
}
```

4.3.4 Update User Role

Change a user's role (User / Admin).

```
PUT /api/v1/identity/users/{userId}/role
Authorization: Bearer <JWT> (Admin)
```

Request Body:

```
{
  "role": "admin"
}
```

Allowed Values: `user` , `admin`

Response (200 OK):

```
{
  "success": true,
  "message": "User role updated to admin",
  "data": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "role": "admin"
  },
  "errors": []
}
```

4.3.5 Get User Statistics

Retrieve aggregated user statistics for the admin dashboard.

```
GET /api/v1/identity/users/stats
Authorization: Bearer <JWT> (Admin)
```

Response (200 OK):

```
{
  "success": true,
  "message": "User statistics retrieved",
  "data": {
    "totalUsers": 1250,
    "activeUsers": 1100,
    "pendingVerification": 120,
    "suspendedUsers": 25,
    "adminUsers": 5
  },
  "errors": []
}
```

5. Card Service API

Base Path: `/api/v1/cards` , `/api/v1/issuers`
Port: 5002
Database: `credvault_cards`

5.1 Card Management Endpoints

5.1.1 List User's Cards

Retrieve all credit cards belonging to the authenticated user.

```
GET /api/v1/cards
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Cards retrieved successfully",
  "data": [
    {
      "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "cardholderName": "JOHN DOE",
      "maskedNumber": "**** **** **** 4444",
      "last4": "4444",
      "expMonth": 12,
      "expYear": 2028,
      "creditLimit": 150000.00,
      "outstandingBalance": 45230.50,
      "isDefault": true,
      "isVerified": true,
      "network": "Visa",
      "issuerName": "HDFC Bank",
      "billingCycleStartDay": 15,
      "createdAt": "2026-02-10T08:00:00Z",
      "updatedAt": "2026-04-15T10:30:00Z"
    }
  ],
  "errors": []
}
```

> **Note:** Soft-deleted cards (`IsDeleted = true`) are automatically excluded from results.

5.1.2 Add New Card

Add a new credit card to the user's account. The card number is encrypted before storage.

```
POST /api/v1/cards
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "cardholderName": "JOHN DOE",
  "cardNumber": "4111222233334444",
  "expMonth": 12,
  "expYear": 2028,
  "issuerId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "creditLimit": 150000.00,
  "billingCycleStartDay": 15,
  "isDefault": true
}
```

Validation Rules:

Field	Type	Required	Constraints
`cardholderName`	string	Yes	Min 2 characters, max 200
`cardNumber`	string	Yes	16 digits, valid Luhn checksum
`expMonth`	int	Yes	1-12
`expYear`	int	Yes	Must be >= current year
`issuerId`	GUID	Yes	Must reference a valid issuer
`creditLimit`	decimal	Yes	Must be > 0
`billingCycleStartDay`	int	Yes	1-28
`isDefault`	boolean	No	Default: false

Response (201 Created):

```
{
  "success": true,
  "message": "Card added successfully",
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "cardholderName": "JOHN DOE",
    "maskedNumber": "**** **** **** 4444",
    "last4": "4444",
    "expMonth": 12,
    "expYear": 2028,
    "creditLimit": 150000.00,
    "outstandingBalance": 0.00,
    "isDefault": true,
    "isVerified": true,
    "network": "Visa",
    "issuerName": "HDFC Bank",
    "billingCycleStartDay": 15,
    "createdAt": "2026-04-30T12:00:00Z"
  },
  "errors": []
}
```

Error Responses:

Status	Condition
400	Validation errors (invalid card number, expired card)
404	Issuer not found
409	Duplicate card already exists for this user

5.1.3 Get Card Details

Retrieve details of a specific card.

```
GET /api/v1/cards/{cardId}
Authorization: Bearer <JWT>
```

Path Parameters:

Parameter	Type	Description
`cardId`	GUID	The card's unique identifier

Response (200 OK):

```
{
  "success": true,
  "message": "Card retrieved successfully",
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "cardholderName": "JOHN DOE",
    "maskedNumber": "**** **** **** 4444",
    "last4": "4444",
    "expMonth": 12,
    "expYear": 2028,
    "creditLimit": 150000.00,
    "outstandingBalance": 45230.50,
    "isDefault": true,
    "isVerified": true,
    "network": "Visa",
    "issuerName": "HDFC Bank",
    "billingCycleStartDay": 15,
    "createdAt": "2026-02-10T08:00:00Z",
    "updatedAt": "2026-04-15T10:30:00Z"
  },
  "errors": []
}
```

Error Responses:

Status	Condition
403	Card does not belong to the authenticated user
404	Card not found or has been deleted

5.1.4 Update Card

Update card details (cardholder name, credit limit, billing cycle).

```
PUT /api/v1/cards/{cardId}
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "cardholderName": "JOHN A. DOE",
  "creditLimit": 200000.00,
  "billingCycleStartDay": 20
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Card updated successfully",
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "cardholderName": "JOHN A. DOE",
    "maskedNumber": "**** **** **** 4444",
    "creditLimit": 200000.00,
    "billingCycleStartDay": 20,
    "updatedAt": "2026-04-30T14:00:00Z"
  },
  "errors": []
}
```

5.1.5 Set Default Card

Mark a card as the user's default card.

```
PATCH /api/v1/cards/{cardId}/default
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Card set as default",
  "data": {
    "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "isDefault": true
  },
  "errors": []
}
```

> **Note:** Setting a card as default automatically unsets the previous default card.

5.1.6 Delete Card (Soft Delete)

Soft-delete a card. The card is marked as `IsDeleted = true` and excluded from queries.

```
DELETE /api/v1/cards/{cardId}
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Card deleted successfully",
  "data": null,
  "errors": []
}
```

> **Note:** This is a soft delete. The card record is retained in the database with `IsDeleted = true`. Physical deletion is not supported.

5.2 Card Transaction Endpoints

5.2.1 Get Card Transactions

Retrieve all transactions for a specific card.

```
GET /api/v1/cards/{cardId}/transactions
Authorization: Bearer <JWT>
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page (max 100)
`type`	string	—	Filter by type: `Purchase`, `Payment`, `Refund`
`dateFrom`	date	—	Filter from date (ISO 8601)
`dateTo`	date	—	Filter to date (ISO 8601)

Response (200 OK):

```
{
  "success": true,
  "message": "Transactions retrieved successfully",
  "data": {
    "items": [
      {
        "id": "b1c2d3e4-f5a6-7890-bcde-f12345678901",
        "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "type": "Purchase",
        "amount": 2500.00,
        "description": "Amazon India - Electronics",
        "date": "2026-04-28T15:30:00Z"
      },
      {
        "id": "c2d3e4f5-a6b7-8901-cdef-123456789012",
        "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "type": "Payment",
        "amount": 15000.00,
        "description": "Bill payment via wallet",
        "date": "2026-04-25T10:00:00Z"
      }
    ],
    "totalCount": 45,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 3
  },
  "errors": []
}
```

5.2.2 Record Card Transaction

Manually record a transaction for a card.

```
POST /api/v1/cards/{cardId}/transactions
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "type": "Purchase",
  "amount": 2500.00,
  "description": "Amazon India - Electronics",
  "date": "2026-04-28T15:30:00Z"
}
```

Allowed Types: `Purchase` , `Payment` , `Refund`

Response (201 Created):

```
{
  "success": true,
  "message": "Transaction recorded successfully",
  "data": {
    "id": "b1c2d3e4-f5a6-7890-bcde-f12345678901",
    "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "type": "Purchase",
    "amount": 2500.00,
    "description": "Amazon India - Electronics",
    "date": "2026-04-28T15:30:00Z"
  },
  "errors": []
}
```

5.2.3 Get All User Transactions

Retrieve all transactions across all of the user's cards.

```
GET /api/v1/cards/transactions
Authorization: Bearer <JWT>
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page (max 100)
`type`	string	—	Filter by transaction type
`dateFrom`	date	—	Filter from date
`dateTo`	date	—	Filter to date

5.2.4 Get Cards by User ID

Retrieve all cards for a specific user (admin or self-lookup).

```
GET /api/v1/cards/user/{userId}
Authorization: Bearer <JWT>
```

5.3 Card Issuer Endpoints

5.3.1 List All Issuers

Retrieve all supported card issuers.

```
GET /api/v1/issuers
```

Response (200 OK):

```
{
  "success": true,
  "message": "Issuers retrieved successfully",
  "data": [
    {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "name": "HDFC Bank",
      "network": "Visa",
      "createdAt": "2026-01-01T00:00:00Z"
    },
    {
      "id": "4fb96g75-6828-5673-c4gd-3d074g77bgb7",
      "name": "ICICI Bank",
      "network": "Mastercard",
      "createdAt": "2026-01-01T00:00:00Z"
    },
    {
      "id": "5gc07h86-7939-6784-d5he-4e185h88chc8",
      "name": "SBI Card",
      "network": "Rupay",
      "createdAt": "2026-01-01T00:00:00Z"
    },
    {
      "id": "6hd18i97-8040-7895-e6if-5f296i99did9",
      "name": "American Express",
      "network": "Amex",
      "createdAt": "2026-01-01T00:00:00Z"
    }
  ],
  "errors": []
}
```

5.3.2 Get Issuer Details

```
GET /api/v1/issuers/{issuerId}
```

5.3.3 Create Issuer (Admin)

```
POST /api/v1/issuers
Authorization: Bearer <JWT> (Admin)
```

Request Body:

```
{
  "name": "Axis Bank",
  "network": "Visa"
}
```

Allowed Networks: `Visa` , `Mastercard` , `Rupay` , `Amex`

5.3.4 Update Issuer (Admin)

```
PUT /api/v1/issuers/{issuerId}
Authorization: Bearer <JWT> (Admin)
```

5.3.5 Delete Issuer (Admin)

```
DELETE /api/v1/issuers/{issuerId}
Authorization: Bearer <JWT> (Admin)
```

5.4 Admin Card Endpoints

5.4.1 List All Cards (Admin)

```
GET /api/v1/cards/admin?pageNumber=1&pageSize=20
Authorization: Bearer <JWT> (Admin)
```

Returns a paginated list of all cards across all users, including soft-deleted cards.

5.4.2 Get Card Details (Admin)

```
GET /api/v1/cards/{cardId}/admin
Authorization: Bearer <JWT> (Admin)
```

5.4.3 Update Card (Admin)

```
PUT /api/v1/cards/{cardId}/admin
Authorization: Bearer <JWT> (Admin)
```

5.4.4 Get Card Transactions (Admin)

```
GET /api/v1/cards/admin/{cardId}/transactions
Authorization: Bearer <JWT> (Admin)
```

6. Billing Service API

Base Path: `/api/v1/billing`
Port: 5003
Database: `credvault_billing`

6.1 Bill Endpoints

6.1.1 List User's Bills

Retrieve all bills for the authenticated user.

```
GET /api/v1/billing/bills
Authorization: Bearer <JWT>
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page (max 100)
`status`	string	—	Filter: `Pending` , `Paid` , `Overdue` , `PartiallyPaid`
`cardId`	GUID	—	Filter by specific card

Response (200 OK):

```
{
  "success": true,
  "message": "Bills retrieved successfully",
  "data": {
    "items": [
      {
        "id": "d3e4f5a6-b7c8-9012-defg-234567890123",
        "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
        "cardLast4": "4444",
        "cardNetwork": "Visa",
        "issuerName": "HDFC Bank",
        "amount": 45230.50,
        "minDue": 4523.05,
        "currency": "INR",
        "billingDate": "2026-04-15T00:00:00Z",
        "dueDate": "2026-05-05T00:00:00Z",
        "amountPaid": 0.00,
        "paidAt": null,
        "status": "Pending",
        "createdAt": "2026-04-15T00:00:00Z",
        "updatedAt": "2026-04-15T00:00:00Z"
      }
    ],
    "totalCount": 12,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 1
  },
  "errors": []
}
```


6.1.2 Get Bill Details

```
GET /api/v1/billing/bills/{billId}
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Bill retrieved successfully",
  "data": {
    "id": "d3e4f5a6-b7c8-9012-defg-234567890123",
    "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "cardLast4": "4444",
    "cardNetwork": "Visa",
    "issuerName": "HDFC Bank",
    "amount": 45230.50,
    "minDue": 4523.05,
    "currency": "INR",
    "billingDate": "2026-04-15T00:00:00Z",
    "dueDate": "2026-05-05T00:00:00Z",
    "amountPaid": 0.00,
    "paidAt": null,
    "status": "Pending",
    "createdAt": "2026-04-15T00:00:00Z",
    "updatedAt": "2026-04-15T00:00:00Z"
  },
  "errors": []
}
```

6.1.3 Check Pending Bill

Check if a specific card has an outstanding pending bill.

```
GET /api/v1/billing/bills/has-pending/{cardId}
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Pending bill check completed",
  "data": {
    "hasPendingBill": true,
    "billId": "d3e4f5a6-b7c8-9012-defg-234567890123",
    "amount": 45230.50,
    "minDue": 4523.05,
    "dueDate": "2026-05-05T00:00:00Z",
    "status": "Pending"
  },
  "errors": []
}
```

6.1.4 Generate Bill (Admin)

Manually trigger bill generation for a card.

```
POST /api/v1/billing/bills/admin/generate-bill
Authorization: Bearer <JWT> (Admin)
```

Request Body:

```
{
  "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Response (201 Created):

```
{
  "success": true,
  "message": "Bill generated successfully",
  "data": {
    "id": "d3e4f5a6-b7c8-9012-defg-234567890123",
    "amount": 45230.50,
    "minDue": 4523.05,
    "billingDate": "2026-04-30T00:00:00Z",
    "dueDate": "2026-05-20T00:00:00Z",
    "status": "Pending"
  },
  "errors": []
}
```

6.1.5 Check Overdue Bills (Admin)

Scan for bills past their due date and mark them as `Overdue`.

```
POST /api/v1/billing/bills/admin/check-overdue
Authorization: Bearer <JWT> (Admin)
```

Response (200 OK):

```
{
  "success": true,
  "message": "Overdue check completed. 3 bills marked as overdue.",
  "data": {
    "billsMarkedOverdue": 3,
    "overdueBillIds": [
      "d3e4f5a6-b7c8-9012-defg-234567890123",
      "e4f5a6b7-c8d9-0123-efgh-345678901234",
      "f5a6b7c8-d9e0-1234-fghi-456789012345"
    ]
  },
  "errors": []
}
```

6.2 Statement Endpoints

6.2.1 List User's Statements

```
GET /api/v1/billing/statements
Authorization: Bearer <JWT>
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page
`cardId`	GUID	—	Filter by card

Response (200 OK):

```
{
  "success": true,
  "message": "Statements retrieved successfully",
  "data": {
    "items": [
      {
        "id": "g6h7i8j9-k0l1-2345-mnop-567890123456",
        "statementPeriod": "April 2026",
        "periodStart": "2026-04-01T00:00:00Z",
        "periodEnd": "2026-04-30T00:00:00Z",
        "openingBalance": 30000.00,
        "totalPurchases": 25000.00,
        "totalPayments": 15000.00,
        "totalRefunds": 2000.00,
        "penaltyCharges": 500.00,
        "interestCharges": 1230.50,
        "closingBalance": 45230.50,
        "minimumDue": 4523.05,
        "amountPaid": 0.00,
        "status": "Generated",
        "cardLast4": "4444",
        "cardNetwork": "Visa",
        "issuerName": "HDFC Bank",
        "creditLimit": 150000.00,
        "availableCredit": 104769.50,
        "createdAt": "2026-04-30T00:00:00Z"
      }
    ],
    "totalCount": 6,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 1
  },
  "errors": []
}
```


6.2.2 Get Statement Details

```
GET /api/v1/billing/statements/{statementId}
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Statement retrieved successfully",
  "data": {
    "id": "g6h7i8j9-k0l1-2345-mnop-567890123456",
    "statementPeriod": "April 2026",
    "periodStart": "2026-04-01T00:00:00Z",
    "periodEnd": "2026-04-30T00:00:00Z",
    "openingBalance": 30000.00,
    "totalPurchases": 25000.00,
    "totalPayments": 15000.00,
    "totalRefunds": 2000.00,
    "penaltyCharges": 500.00,
    "interestCharges": 1230.50,
    "closingBalance": 45230.50,
    "minimumDue": 4523.05,
    "amountPaid": 0.00,
    "status": "Generated",
    "cardLast4": "4444",
    "cardNetwork": "Visa",
    "issuerName": "HDFC Bank",
    "creditLimit": 150000.00,
    "availableCredit": 104769.50,
    "createdAt": "2026-04-30T00:00:00Z",
    "updatedAt": "2026-04-30T00:00:00Z"
  },
  "errors": []
}
```


6.2.3 Get Statement Transactions

Retrieve the detailed transaction breakdown for a statement.

```
GET /api/v1/billing/statements/{statementId}/transactions
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Statement transactions retrieved successfully",
  "data": {
    "items": [
      {
        "id": "h7i8j9k0-l1m2-3456-nopq-678901234567",
        "sourceTransactionId": "b1c2d3e4-f5a6-7890-bcde-f12345678901",
        "type": "Purchase",
        "amount": 2500.00,
        "description": "Amazon India - Electronics",
        "date": "2026-04-28T15:30:00Z"
      }
    ],
    "totalCount": 25,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 2
  },
  "errors": []
}
```


6.3 Rewards Endpoints

6.3.1 Get Reward Account

Retrieve the user's rewards account including current points balance and tier information.

```
GET /api/v1/billing/rewards/account
Authorization: Bearer <JWT>
```

Response (200 OK):


```
{
  "success": true,
  "message": "Reward account retrieved successfully",
  "data": {
    "id": "i8j9k0l1-m2n3-4567-opqr-789012345678",
    "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "pointsBalance": 12500.00,
    "tierName": "Gold",
    "rewardRate": 1.5,
    "cardNetwork": "Visa",
    "issuerName": "HDFC Bank",
    "createdAt": "2026-02-10T08:00:00Z",
    "updatedAt": "2026-04-28T15:30:00Z"
  },
  "errors": []
}
```

6.3.2 Get Reward Transaction History

```
GET /api/v1/billing/rewards/transactions
Authorization: Bearer <JWT>
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page
`type`	string	—	Filter: `Earned` , `Redeemed` , `Reversed`

Response (200 OK):

```
{
  "success": true,
  "message": "Reward transactions retrieved successfully",
  "data": {
    "items": [
      {
        "id": "j9k0l1m2-n3o4-5678-pqrs-890123456789",
        "billId": "d3e4f5a6-b7c8-9012-defg-234567890123",
        "points": 750.00,
        "type": "Earned",
        "createdAt": "2026-04-28T16:00:00Z"
      },
      {
        "id": "k0l1m2n3-o4p5-6789-qrst-901234567890",
        "billId": "c2d3e4f5-a6b7-8901-cdef-123456789012",
        "points": 500.00,
        "type": "Redeemed",
        "createdAt": "2026-04-25T10:30:00Z"
      }
    ],
    "totalCount": 15,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 1
  },
  "errors": []
}
```

6.3.3 List Reward Tiers

```
GET /api/v1/billing/rewards/tiers
```

Response (200 OK):

```
{
  "success": true,
  "message": "Reward tiers retrieved successfully",
  "data": [
    {
      "id": "l1m2n3o4-p5q6-7890-rstu-012345678901",
      "cardNetwork": "Visa",
      "issuerId": null,
      "issuerName": "All Issuers",
      "minSpend": 0.00,
      "rewardRate": 1.0,
      "effectiveFrom": "2026-01-01T00:00:00Z",
      "effectiveTo": null,
      "createdAt": "2026-01-01T00:00:00Z"
    },
    {
      "id": "m2n3o4p5-q6r7-8901-stuv-123456789012",
      "cardNetwork": "Visa",
      "issuerId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "issuerName": "HDFC Bank",
      "minSpend": 50000.00,
      "rewardRate": 1.5,
      "effectiveFrom": "2026-01-01T00:00:00Z",
      "effectiveTo": null,
      "createdAt": "2026-01-01T00:00:00Z"
    }
  ],
  "errors": []
}
```

6.3.4 Create Reward Tier (Admin)

```
POST /api/v1/billing/rewards/tiers
Authorization: Bearer <JWT> (Admin)
```

Request Body:

```
{
  "cardNetwork": "Mastercard",
  "issuerId": "4fb96g75-6828-5673-c4gd-3d074g77bgb7",
  "minSpend": 25000.00,
  "rewardRate": 1.25,
  "effectiveFrom": "2026-05-01T00:00:00Z",
  "effectiveTo": null
}
```

6.3.5 Update Reward Tier (Admin)

```
PUT /api/v1/billing/rewards/tiers/{tierId}
Authorization: Bearer <JWT> (Admin)
```

6.3.6 Delete Reward Tier (Admin)

```
DELETE /api/v1/billing/rewards/tiers/{tierId}
Authorization: Bearer <JWT> (Admin)
```

6.4 Internal Endpoints (Saga)

6.4.1 Redeem Rewards (Internal)

Used by the Payment Service saga to redeem rewards during payment processing. Not intended for direct client consumption.

```
POST /api/v1/billing/rewards/internal/redeem
Authorization: Internal Service Token
```

Request Body:

```
{
  "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "pointsToRedeem": 500.00,
  "billId": "d3e4f5a6-b7c8-9012-defg-234567890123",
  "correlationId": "n3o4p5q6-r7s8-9012-tuvw-234567890123"
}
```

7. Payment Service API

Base Path: `/api/v1/payments` , `/api/v1/wallets`
Port: 5004
Database: `credvault_payments`

7.1 Payment Endpoints

7.1.1 Initiate Payment

Start the payment flow. Validates the bill, creates a payment record, generates an OTP, and sends it via email.

```
POST /api/v1/payments/initiate
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "cardId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "billId": "d3e4f5a6-b7c8-9012-defg-234567890123",
  "amount": 45230.50,
  "paymentType": "Wallet",
  "rewardsAmount": 500.00
}
```

Validation Rules:

Field	Type	Required	Constraints
`cardId`	GUID	Yes	Must belong to authenticated user
`billId`	GUID	Yes	Must be Pending or PartiallyPaid
`amount`	decimal	Yes	Must be >= minimum due
`paymentType`	string	Yes	`Wallet` or `Card`
`rewardsAmount`	decimal	No	Must be <= available reward points

Response (201 Created):

```
{
  "success": true,
  "message": "Payment initiated. Please verify with the OTP sent to your email.",
  "data": {
    "paymentId": "o4p5q6r7-s8t9-0123-uvwx-345678901234",
    "amount": 45230.50,
    "paymentType": "Wallet",
    "rewardsAmount": 500.00,
    "status": "Initiated",
    "otpExpiresAt": "2026-04-30T12:10:00Z",
    "createdAt": "2026-04-30T12:05:00Z"
  },
  "errors": []
}
```

Error Responses:

Status	Condition
400	Validation errors
404	Bill or card not found

Status	Condition
422	Insufficient wallet balance (if Wallet payment)
422	Bill already paid
422	RewardsAmount exceeds available points

7.1.2 Verify Payment OTP

Verify the OTP and trigger the payment saga orchestration.

```
POST /api/v1/payments/{paymentId}/verify-otp
Authorization: Bearer <JWT>
```

Path Parameters:

Parameter	Type	Description
`paymentId`	GUID	The payment's unique identifier

Request Body:

```
{
  "otp": "654321"
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "OTP verified. Payment is being processed.",
  "data": {
    "paymentId": "o4p5q6r7-s8t9-0123-uvwx-345678901234",
    "status": "Processing"
  },
  "errors": []
}
```

Error Responses:

Status	Condition
400	Invalid OTP format
401	Invalid or expired OTP
404	Payment not found

> **Note:** After OTP verification, the payment saga executes asynchronously. The client should poll the payment status or use WebSocket for real-time updates.

7.1.3 Resend Payment OTP

Request a new OTP for an existing payment.

```
POST /api/v1/payments/{paymentId}/resend-otp
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Payment OTP has been resent to your email.",
  "data": {
    "paymentId": "o4p5q6r7-s8t9-0123-uvwx-345678901234",
    "otpExpiresAt": "2026-04-30T12:20:00Z"
  },
  "errors": []
}
```

7.1.4 Get Payment Transactions

Retrieve the transaction history for a specific payment.

```
GET /api/v1/payments/{paymentId}/transactions
Authorization: Bearer <JWT>
```

Response (200 OK):

```json
{
  "success": true,
  "message": "Payment transactions retrieved successfully",
  "data": {
    "payment": {
      "id": "o4p5q6r7-s8t9-0123-uvwx-345678901234",
      "amount": 45230.50,
      "paymentType": "Wallet",
      "status": "Paid",
      "createdAt": "2026-04-30T12:05:00Z",
      "updatedAt": "2026-04-30T12:07:00Z"
    },
    "transactions": [
      {
        "id": "p5q6r7s8-t9u0-1234-vwxy-456789012345",
        "amount": 45230.50,
        "type": "Debit",
        "description": "Bill payment - April 2026",
        "createdAt": "2026-04-30T12:07:00Z"
      }
    ]
  },
  "errors": []
}
```

7.2 Wallet Endpoints

7.2.1 Get Wallet Info

Retrieve the authenticated user's wallet information.

```
GET /api/v1/wallets/me
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Wallet retrieved successfully",
  "data": {
    "id": "q6r7s8t9-u0v1-2345-wxyz-567890123456",
    "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "balance": 25000.00,
    "totalTopUps": 50000.00,
    "totalSpent": 25000.00,
    "createdAt": "2026-02-10T08:00:00Z",
    "updatedAt": "2026-04-30T12:07:00Z"
  },
  "errors": []
}
```

> **Note:** Wallet is auto-created on first use. If no wallet exists, this endpoint will return an error or auto-initialize depending on implementation.

7.2.2 Get Wallet Balance

Retrieve only the wallet balance (lightweight endpoint).

```
GET /api/v1/wallets/balance
Authorization: Bearer <JWT>
```

Response (200 OK):

```
{
  "success": true,
  "message": "Balance retrieved successfully",
  "data": {
    "balance": 25000.00,
    "currency": "INR"
  },
  "errors": []
}
```

7.2.3 Get Wallet Transaction History

```
GET /api/v1/wallets/transactions
Authorization: Bearer <JWT>
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page (max 100)
`type`	string	—	Filter: `TopUp` , `Debit` , `Refund`

Response (200 OK):

```
{
  "success": true,
  "message": "Wallet transactions retrieved successfully",
  "data": {
    "items": [
      {
        "id": "r7s8t9u0-v1w2-3456-xyza-678901234567",
        "type": "TopUp",
        "amount": 10000.00,
        "balanceAfter": 25000.00,
        "description": "Razorpay wallet top-up",
        "relatedPaymentId": null,
        "createdAt": "2026-04-28T09:00:00Z"
      },
      {
        "id": "s8t9u0v1-w2x3-4567-yzab-789012345678",
        "type": "Debit",
        "amount": 45230.50,
        "balanceAfter": 15000.00,
        "description": "Bill payment - April 2026",
        "relatedPaymentId": "o4p5q6r7-s8t9-0123-uvwx-345678901234",
        "createdAt": "2026-04-30T12:07:00Z"
      }
    ],
    "totalCount": 30,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 2
  },
  "errors": []
}
```

7.2.4 Razorpay Wallet Top-Up

Initiate a wallet top-up via Razorpay payment gateway.

```
POST /api/v1/wallets/topup
Authorization: Bearer <JWT>
```

Request Body:

```
{
  "amount": 10000.00
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Razorpay order created. Proceed with payment.",
  "data": {
    "razorpayOrderId": "order_NXXXXXXXXXXXXX",
    "amount": 10000.00,
    "currency": "INR",
    "topUpId": "t9u0v1w2-x3y4-5678-zabc-890123456789"
  },
  "errors": []
}
```

> **Note:** The frontend should use the returned `razorpayOrderId` to open the Razorpay Checkout widget. Payment completion is handled via webhook callback.

7.2.5 Razorpay Webhook Handler

Receives payment callbacks from Razorpay. Verifies signature and updates wallet balance.

```
POST /api/v1/wallets/razorpay-webhook
```

Headers:

Header	Description
`X-Razorpay-Signature`	HMAC-SHA256 signature for verification

Request Body (Razorpay payload):

```
{
  "event": "payment.captured",
  "contains": ["payment"],
  "payment": {
    "entity": "payment",
    "id": "pay_NXXXXXXXXXXXXX",
    "order_id": "order_NXXXXXXXXXXXXX",
    "amount": 1000000,
    "currency": "INR",
    "status": "captured"
  }
}
```

Response (200 OK):

```
{
  "success": true,
  "message": "Webhook processed successfully",
  "data": null,
  "errors": []
}
```

8. Notification Service API

Base Path: `/api/v1/notifications`
Port: 5005
Database: `credvault_notifications`

> **Note:** All Notification Service endpoints require Admin authentication. This service is primarily event-driven; its API is for auditing and log inspection.

8.1 Notification Logs

8.1.1 Get Notification Logs

Retrieve the history of sent notifications with delivery status.

```
GET /api/v1/notifications/logs
Authorization: Bearer <JWT> (Admin)
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page
`type`	string	—	Filter: `Email` , `SMS`
`isSuccess`	boolean	—	Filter by delivery status
`userId`	GUID	—	Filter by recipient user
`dateFrom`	date	—	Filter from date
`dateTo`	date	—	Filter to date

Response (200 OK):

```
{
  "success": true,
  "message": "Notification logs retrieved successfully",
  "data": {
    "items": [
      {
        "id": "u0v1w2x3-y4z5-6789-abcd-901234567890",
        "userId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "recipient": "john@example.com",
        "subject": "Your CredVault Verification OTP",
        "type": "Email",
        "isSuccess": true,
        "errorMessage": null,
        "traceId": "0HN12345678ABC",
        "createdAt": "2026-04-30T12:00:00Z"
      }
    ],
    "totalCount": 500,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 25
  },
  "errors": []
}
```


8.2 Audit Trail

8.2.1 Get Audit Logs

Retrieve the system-wide audit trail of administrative actions.

```
GET /api/v1/notifications/audit
Authorization: Bearer <JWT> (Admin)
```

Query Parameters:

Parameter	Type	Default	Description
`pageNumber`	int	1	Page number
`pageSize`	int	20	Items per page
`entityName`	string	—	Filter by entity type (User, Card, Bill, etc.)
`action`	string	—	Filter by action (Create, Update, Delete, etc.)
`userId`	GUID	—	Filter by acting user
`dateFrom`	date	—	Filter from date
`dateTo`	date	—	Filter to date

Response (200 OK):

```
{
  "success": true,
  "message": "Audit logs retrieved successfully",
  "data": {
    "items": [
      {
        "id": "v1w2x3y4-z5a6-7890-bcde-012345678901",
        "entityName": "User",
        "entityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "action": "UpdateRole",
        "userId": "admin-user-id-here",
        "changes": "{\"role\": {\"old\": \"user\", \"new\": \"admin\"}}",
        "traceId": "0HN12345678ABC",
        "createdAt": "2026-04-30T14:00:00Z"
      }
    ],
    "totalCount": 200,
    "pageNumber": 1,
    "pageSize": 20,
    "totalPages": 10
  },
  "errors": []
}
```

9. Error Handling

9.1 Error Response Format

All errors follow the standard `ApiResponse<T>` envelope:

```
{
  "success": false,
  "message": "A human-readable error description",
  "data": null,
  "errors": ["Specific error detail 1", "Specific error detail 2"],
  "traceId": "0HN12345678ABC"
}
```

9.2 Error Categories

Validation Errors (400)

Returned when request data fails FluentValidation rules:

```
{
  "success": false,
  "message": "Validation failed",
  "data": null,
  "errors": [
    "Email is required",
    "Password must be at least 8 characters",
    "Password must contain at least one uppercase letter",
    "Password must contain at least one digit"
  ],
  "traceId": "0HN12345678ABC"
}
```

Authentication Errors (401)

```
{
  "success": false,
  "message": "Unauthorized. Invalid or expired token.",
  "data": null,
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

Authorization Errors (403)

```
{
  "success": false,
  "message": "Forbidden. Admin role required.",
  "data": null,
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

Not Found Errors (404)

```
{
  "success": false,
  "message": "Card with ID 'a1b2c3d4-e5f6-7890-abcd-ef1234567890' was not found.",
  "data": null,
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

Business Logic Errors (422)

```
{
  "success": false,
  "message": "Insufficient wallet balance. Required: 45230.50, Available: 25000.00",
  "data": null,
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

Server Errors (500)

```
{
  "success": false,
  "message": "An internal server error occurred. Please try again later.",
  "data": null,
  "errors": [],
  "traceId": "0HN12345678ABC"
}
```

> **Security Note:** Internal error details (stack traces, exception types) are never exposed to clients. They are logged server-side with Serilog.

9.3 Exception Handling Middleware

All services use a shared `ExceptionHandlingMiddleware` that:

- Catches all unhandled exceptions
- Maps exception types to appropriate HTTP status codes
- Logs the full exception with Serilog (including correlation ID, machine name)
- Returns a sanitized error response to the client

9.4 Client-Side Error Handling

The Angular `authInterceptor` handles 401 responses automatically:

- Clears `sessionStorage` (`cv_token` , `cv_user`)
- Redirects to `/login`

For other errors, the frontend displays the `message` and `errors` fields from the response.

10. Rate Limiting & Throttling

10.1 OTP Rate Limits

Operation	Limit	Window	Lockout
Registration OTP	3 attempts	10 minutes	Cooldown period

Operation	Limit	Window	Lockout
Password Reset OTP	3 attempts	10 minutes	Cooldown period
Payment OTP	3 attempts	5 minutes	Payment expires

10.2 Login Rate Limits

Operation	Limit	Window	Response
Login attempts	5 failures	15 minutes	Temporary lockout

10.3 API Rate Limits

Endpoint Category	Rate Limit	Window
Public (auth)	30 req/min	Per IP
Authenticated	100 req/min	Per user
Admin	200 req/min	Per admin user

> **Note:** Rate limiting is configurable via Ocelot Gateway middleware. Current implementation uses in-memory rate limiting suitable for single-instance deployments.

11. Health Checks

11.1 Health Check Endpoints

Service	URL	Checks
API Gateway	`GET /health`	Gateway status
Identity Service	`GET /api/v1/identity/health`	DB connectivity
Card Service	`GET /api/v1/cards/health`	DB connectivity
Billing Service	`GET /api/v1/billing/health`	DB connectivity
Payment Service	`GET /api/v1/payments/health`	DB connectivity
Notification Service	`GET /api/v1/notifications/health`	DB connectivity, SMTP

11.2 Health Response Format

```
{
  "status": "Healthy",
  "totalDuration": "00:00:00.1234567",
  "entries": {
    "database": {
      "status": "Healthy",
      "duration": "00:00:00.0456789"
    }
  }
}
```

11.3 Smoke Test

```
# Verify full stack connectivity via Gateway
curl -s -o /dev/null -w "%{http_code}" \
  http://localhost:5006/health
# Expected: 200

# Verify Identity Service health
curl -s http://localhost:5006/api/v1/identity/health \
  -H "Accept: application/json"
# Expected: {"status": "Healthy", ...}
```

12. Frontend Integration Guide

12.1 Angular Service Mapping

Angular Service	Method	HTTP Method	Endpoint
`AuthService`	`register()`	POST	`/api/v1/identity/auth/register`
`AuthService`	`login()`	POST	`/api/v1/identity/auth/login`
`AuthService`	`googleLogin()`	POST	`/api/v1/identity/auth/google`
`AuthService`	`verifyEmailOtp()`	POST	`/api/v1/identity/auth/verify-email-otp`
`AuthService`	`resendVerification()`	POST	`/api/v1/identity/auth/resend-verification`
`AuthService`	`forgotPassword()`	POST	`/api/v1/identity/auth/forgot-password`
`AuthService`	`resetPassword()`	POST	`/api/v1/identity/auth/reset-password`
`AuthService`	`getProfile()`	GET	`/api/v1/identity/users/me`
`AuthService`	`updateProfile()`	PUT	`/api/v1/identity/users/me`

Angular Service	Method	HTTP Method	Endpoint
AuthService	changePassword()	PUT	/api/v1/identity/users/me/password
DashboardService	getCards()	GET	/api/v1/cards
DashboardService	addCard()	POST	/api/v1/cards
DashboardService	updateCard()	PUT	/api/v1/cards/{id}
DashboardService	deleteCard()	DELETE	/api/v1/cards/{id}
DashboardService	getCardTransactions()	GET	/api/v1/cards/{cardId}/transactions
BillingService	getBills()	GET	/api/v1/billing/bills
BillingService	getBill()	GET	/api/v1/billing/bills/{id}
BillingService	hasPendingBill()	GET	/api/v1/billing/bills/has-pending/{cardId}
StatementService	getStatements()	GET	/api/v1/billing/statements
StatementService	getStatement()	GET	/api/v1/billing/statements/{id}
StatementService	getStatementTransactions()	GET	/api/v1/billing/statements/{id}/transactions
RewardsService	getRewardAccount()	GET	/api/v1/billing/rewards/account
RewardsService	getRewardTransactions()	GET	/api/v1/billing/rewards/transactions
RewardsService	getRewardTiers()	GET	/api/v1/billing/rewards/tiers
PaymentService	initiatePayment()	POST	/api/v1/payments/initiate
PaymentService	verifyPaymentOtp()	POST	/api/v1/payments/{id}/verify-otp
PaymentService	resendPaymentOtp()	POST	/api/v1/payments/{id}/resend-otp
PaymentService	getPaymentTransactions()	GET	/api/v1/payments/{id}/transactions
WalletService	getWallet()	GET	/api/v1/wallets/me
WalletService	getBalance()	GET	/api/v1/wallets/balance
WalletService	getTransactions()	GET	/api/v1/wallets/transactions
WalletService	initiateTopUp()	POST	/api/v1/wallets/topup
AdminService	getUsers()	GET	/api/v1/identity/users
AdminService	getUser()	GET	/api/v1/identity/users/{id}
AdminService	updateUserStatus()	PUT	/api/v1/identity/users/{id}/status
AdminService	updateUserRole()	PUT	/api/v1/identity/users/{id}/role
AdminService	getUserStats()	GET	/api/v1/identity/users/stats
AdminService	getAllCards()	GET	/api/v1/cards/admin

Angular Service	Method	HTTP Method	Endpoint
`AdminService`	`generateBill()`	POST	`/api/v1/billing/bills/admin/generate-bill`
`AdminService`	`checkOverdueBills()`	POST	`/api/v1/billing/bills/admin/check-overdue`
`AdminService`	`createRewardTier()`	POST	`/api/v1/billing/rewards/tiers`
`AdminService`	`updateRewardTier()`	PUT	`/api/v1/billing/rewards/tiers/{id}`
`AdminService`	`deleteRewardTier()`	DELETE	`/api/v1/billing/rewards/tiers/{id}`
`AdminService`	`getNotificationLogs()`	GET	`/api/v1/notifications/logs`
`AdminService`	`getAuditLogs()`	GET	`/api/v1/notifications/audit`

12.2 Auth Interceptor Behavior

The Angular `authInterceptor` applies to all HTTP requests:

```
// Pseudocode
intercept(request: HttpRequest, next: HttpHandler): Observable<HttpEvent> {
  const token = sessionStorage.getItem('cv_token');
  if (token) {
    request = request.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }
  return next.handle(request).pipe(
    catchError(error => {
      if (error.status === 401) {
        sessionStorage.removeItem('cv_token');
        sessionStorage.removeItem('cv_user');
        router.navigate(['/login']);
      }
      return throwError(error);
    })
  );
}
```

12.3 Environment Configuration

```
// environments/environment.ts
export const environment = {
  production: false,
  apiUrl: 'http://localhost:5006',
  googleClientId: '<google-client-id>',
  razorpayKeyId: '<razorpay-key-id>'
};
```

For Docker deployments, runtime environment variables are generated via `scripts/generate-runtime-env.js` .

13. Core Business Flows

13.1 User Registration & Verification

```
POST /api/v1/identity/auth/register
  → Creates user (PendingVerification)
  → Publishes IUserRegistered + IUserOtpGenerated to RabbitMQ
  → Notification Service sends OTP email
  → Returns userId and status

POST /api/v1/identity/auth/verify-email-otp
  → Validates OTP (6 digits, not expired)
  → Marks user as Active
  → Issues JWT token
  → Returns token + user profile
```

13.2 Bill Payment with Saga Orchestration

```
POST /api/v1/payments/initiate
  → Validates bill status (Pending/PartiallyPaid)
  → Validates amount >= minimum due
  → Validates wallet balance (if Wallet payment)
  → Creates Payment record (Initiated)
  → Generates 6-digit OTP (5-min expiry)
  → Publishes IPaymentOtpGenerated → Email sent
  → Returns paymentId

POST /api/v1/payments/{id}/verify-otp
  → Validates OTP
  → Publishes IStartPaymentOrchestration to RabbitMQ
  → Returns "Processing" status

  [Saga executes asynchronously via MassTransit state machine]
  → AwaitingBillUpdate → Billing Service updates bill
  → AwaitingRewardRedemption → Billing Service redeems points
  → AwaitingCardDeduction → Card Service deducts balance
  → Completed → Payment marked as Paid
  → On failure → Compensation rollback → Payment marked Compensated
```

13.3 Wallet Top-Up via Razorpay

```
POST /api/v1/wallets/topup
  → Creates Razorpay order
  → Returns razorpayOrderId to frontend

[Frontend opens Razorpay Checkout with orderId]

[Razorpay sends webhook callback]
POST /api/v1/wallets/razorpay-webhook
  → Verifies HMAC-SHA256 signature
  → Updates wallet balance
  → Creates WalletTransaction (TopUp)
  → Marks RazorpayWalletTopUp as Completed
```

13.4 Google OAuth Login

```
POST /api/v1/identity/auth/google
  → Receives Google IdToken
  → Validates IdToken with Google's public keys
  → If user exists (by email) → login
  → If user doesn't exist → create account (Active, no email verification needed)
  → Issues JWT token
  → Returns token + user profile
```

14. Inter-Service Event Reference

14.1 Event Routing Table

Event	Publisher	Consumer(s)	Purpose
`IUserRegistered`	Identity Service	Notification Service	Send welcome email
`IUserOtpGenerated`	Identity Service	Notification Service	Send OTP email
`ICardAdded`	Card Service	Notification Service	Send card confirmation email
`IPaymentOtpGenerated`	Payment Service	Notification Service	Send payment OTP email
`IStartPaymentOrchestration`	Payment Service	Payment Service (Saga)	Trigger saga state machine
`IStartBillUpdate`	Payment Service (Saga)	Billing Service	Update bill status
`IBillUpdated`	Billing Service	Payment Service (Saga)	Confirm bill update
`IStartRewardRedemption`	Payment Service (Saga)	Billing Service	Redeem reward points
`IRewardsRedeemed`	Billing Service	Payment Service (Saga)	Confirm reward redemption
`IStartCardDeduction`	Payment Service (Saga)	Card Service	Deduct card balance

Event	Publisher	Consumer(s)	Purpose
`ICardDeducted`	Card Service	Payment Service (Saga)	Confirm card deduction
`IPaymentCompleted`	Payment Service (Saga)	Notification Service	Send payment success email
`IPaymentCompensated`	Payment Service (Saga)	Notification Service	Send payment failure email

14.2 Event Delivery Guarantees

Mechanism	Description
InMemoryOutbox	MassTransit outbox prevents lost messages during service restart
Retry Policy	Exponential backoff: 1s → 5s → 15s on all consumers
Idempotency	Saga uses CorrelationId; duplicate messages ignored for completed sagas
Dead Letter	Failed messages routed to RabbitMQ dead letter exchange for inspection

15. Appendix

15.1 Enum Reference

Enum	Values
UserStatus	`PendingVerification` (0), `Active` (1), `Suspended` (2)
UserRole	`User` (0), `Admin` (1)
CardNetwork	`Visa` (0), `Mastercard` (1), `Rupay` (2), `Amex` (3)
CardTransactionType	`Purchase` (0), `Payment` (1), `Refund` (2)
BillStatus	`Pending` (0), `Paid` (1), `Overdue` (2), `PartiallyPaid` (3)
StatementStatus	`Generated` (0), `Finalized` (1)
RewardTransactionType	`Earned` (0), `Redeemed` (1), `Reversed` (2)
PaymentType	`Wallet` (0), `Card` (1)
PaymentStatus	`Initiated` (0), `Processing` (1), `Paid` (2), `Failed` (3), `Compensated` (4)
TransactionType	`Debit` (0), `Credit` (1)
WalletTransactionType	`TopUp` (0), `Debit` (1), `Refund` (2)
RazorpayTopUpStatus	`Pending` (0), `Completed` (1), `Failed` (2)
NotificationType	`Email` (0), `SMS` (1)

15.2 Complete Endpoint Inventory (58 Endpoints)

Identity Service (15 endpoints)

#	Method	Endpoint	Auth	Description
1	POST	`/auth/register`	Public	Register new user
2	POST	`/auth/login`	Public	Email/password login
3	POST	`/auth/google`	Public	Google OAuth login
4	POST	`/auth/verify-email-otp`	Public	Verify email with OTP
5	POST	`/auth/resend-verification`	Public	Resend verification OTP
6	POST	`/auth/forgot-password`	Public	Request password reset
7	POST	`/auth/reset-password`	Public	Reset password with OTP
8	GET	`/users/me`	User	Get current profile
9	PUT	`/users/me`	User	Update profile
10	PUT	`/users/me/password`	User	Change password
11	GET	`/users`	Admin	List all users (paginated)
12	GET	`/users/{id}`	Admin	Get user by ID
13	PUT	`/users/{id}/status`	Admin	Update user status
14	PUT	`/users/{id}/role`	Admin	Update user role
15	GET	`/users/stats`	Admin	User statistics

Card Service (19 endpoints)

#	Method	Endpoint	Auth	Description
16	GET	`/cards`	User	List user's cards
17	POST	`/cards`	User	Add new card
18	GET	`/cards/{id}`	User	Get card details
19	PUT	`/cards/{id}`	User	Update card
20	PATCH	`/cards/{id}/default`	User	Set card as default
21	DELETE	`/cards/{id}`	User	Soft-delete card
22	GET	`/cards/{cardId}/transactions`	User	Get card transactions
23	POST	`/cards/{cardId}/transactions`	User	Record card transaction
24	GET	`/cards/transactions`	User	All transactions for user
25	GET	`/cards/user/{userId}`	User	Get cards by user ID

#	Method	Endpoint	Auth	Description
26	GET	`/issuers`	User	List all issuers
27	GET	`/issuers/{id}`	User	Get issuer details
28	POST	`/issuers`	Admin	Create issuer
29	PUT	`/issuers/{id}`	Admin	Update issuer
30	DELETE	`/issuers/{id}`	Admin	Delete issuer
31	GET	`/cards/admin`	Admin	List all cards (paginated)
32	GET	`/cards/{id}/admin`	Admin	Get card details (admin)
33	PUT	`/cards/{id}/admin`	Admin	Update card (admin)
34	GET	`/cards/admin/{id}/transactions`	Admin	Get card transactions (admin)

Billing Service (15 endpoints)

#	Method	Endpoint	Auth	Description
35	GET	`/bills`	User	List user's bills
36	GET	`/bills/{id}`	User	Get bill details
37	GET	`/bills/has-pending/{cardId}`	User	Check pending bill
38	POST	`/bills/admin/generate-bill`	Admin	Generate bill
39	POST	`/bills/admin/check-overdue`	Admin	Check overdue bills
40	GET	`/statements`	User	List statements
41	GET	`/statements/{id}`	User	Get statement details
42	GET	`/statements/{id}/transactions`	User	Statement transactions
43	GET	`/rewards/account`	User	Get reward account
44	GET	`/rewards/transactions`	User	Reward transaction history
45	POST	`/rewards/internal/redeem`	Internal	Redeem rewards (saga)
46	GET	`/rewards/tiers`	User	List reward tiers
47	POST	`/rewards/tiers`	Admin	Create reward tier
48	PUT	`/rewards/tiers/{id}`	Admin	Update reward tier
49	DELETE	`/rewards/tiers/{id}`	Admin	Delete reward tier

Payment Service (9 endpoints)

#	Method	Endpoint	Auth	Description
50	POST	`/payments/initiate`	User	Initiate payment (triggers OTP)

#	Method	Endpoint	Auth	Description
51	POST	`/payments/{id}/verify-otp`	User	Verify payment OTP
52	POST	`/payments/{id}/resend-otp`	User	Resend payment OTP
53	GET	`/payments/{id}/transactions`	User	Payment transactions
54	GET	`/wallets/me`	User	Get wallet info
55	GET	`/wallets/balance`	User	Get wallet balance
56	GET	`/wallets/transactions`	User	Wallet transaction history
57	POST	`/wallets/topup`	User	Initiate Razorpay top-up
58	POST	`/wallets/razorpay-webhook`	Public	Razorpay webhook handler

Notification Service (2 endpoints)

#	Method	Endpoint	Auth	Description
59	GET	`/logs`	Admin	Notification logs
60	GET	`/audit`	Admin	Audit trail

15.3 Versioning & Deprecation Policy

Policy	Detail
Current Version	`v1`
Versioning Strategy	URL-based (`/api/v1/` , `/api/v2/`)
Breaking Changes	Introduced via new major version path
Deprecation Notice	`Warning: 299 - "Deprecated"` header on deprecated endpoints
Sunset Period	Minimum 90 days notice before endpoint removal
Migration Guide	Published alongside new version release

15.4 Security Best Practices

Practice	Implementation
JWT Storage	`sessionStorage` (cleared on browser close)
Token Expiry	60 minutes
Password Hashing	BCrypt
Card Number Storage	AES encrypted; only last 4 digits stored plain
OTP	6 digits, time-limited, single-use
HTTPS	Required for all external communication

Practice	Implementation
CORS	Configured per environment; production restricts to allowed origins
Input Validation	FluentValidation on all request DTOs
SQL Injection	Prevented via EF Core parameterized queries
XSS	Angular built-in sanitization
CSRF	Stateless JWT eliminates CSRF risk

End of CredVault API Reference Documentation

Document Version: 1.0 | Last Updated: April 2026 | Maintained by: Platform Engineering Team